

# The High-Level Synthesis-Based Firmware Upgrade for the Digital Trigger Back-end System of the ATLAS Liquid Argon Calorimeter

1<sup>st</sup> Melissa S. Aguiar

*Department of Electrical Engineering  
Federal University of Juiz de Fora  
Juiz de Fora, Brazil  
melissa.aguiar@cern.ch*

2<sup>nd</sup> Dabson F. Santos

*Department of Electrical Engineering  
Federal University of Juiz de Fora  
Juiz de Fora, Brazil  
dabson.santos@estudante.ufjf.br*

4<sup>th</sup> Marcos V. S. Oliveira

*Physics Department  
Brookhaven National Laboratory  
New Jersey, United States of America  
mo@bnl.gov*

5<sup>th</sup> Luciano M. A. Filho

*Department of Electrical Engineering  
Federal University of Juiz de Fora  
Juiz de Fora, Brazil  
luciano.andrade@engenharia.ufjf.br*

**Abstract**—The trigger and data acquisition systems of the LHC experiments rely on reconfigurable architectures based on FPGA (Field Programmable Gate Arrays) to accommodate design updates and increasing of the detectors complexity. However, this flexibility comes at the cost of longer development and validation cycles, particularly for large real-time systems operating under strict latency and throughput constraints. This work presents the HLS-based firmware redesign of the back-end system from the ATLAS Liquid Argon Calorimeter, where the introduction of a parallel switch matrix on REMAP block significantly simplifies the architecture and reduces the use of firmware resources. The proposed methodology allows systematic design exploration without modifying the source code, shortens verification time, and provides a more efficient workflow for creating and maintaining large-scale FPGA firmware in modern trigger systems.

**Index Terms**—HLS, Real-time Trigger Systems, FPGA.

## I. INTRODUCTION

The Large Hadron Collider (LHC) at CERN is currently the world's largest particle accelerator, designed to study proton–proton collisions at very high energies [1]. The ATLAS experiment is one of its general-purpose detectors, built to explore a wide range of physics, from precision measurements to searches for new particles [2]. Surrounding the particle's collision point in a layered structure, ATLAS is composed of three principal subsystems: the Inner Detector, the Muon Spectrometer and the Calorimeters.

Within the calorimeter system (Fig. 1), the Tile Calorimeter [3] measures the energy of particles that interact hadronically with its material, while the Liquid Argon Calorimeter [4] provides precision measurements of electromagnetic energy deposits from electrons and photons. These measurements are essential inputs to the ATLAS Trigger and Data Acquisition (TDAQ) system, which reduces the 40 MHz LHC rate to keep only the relevant events for storage and analysis [5].

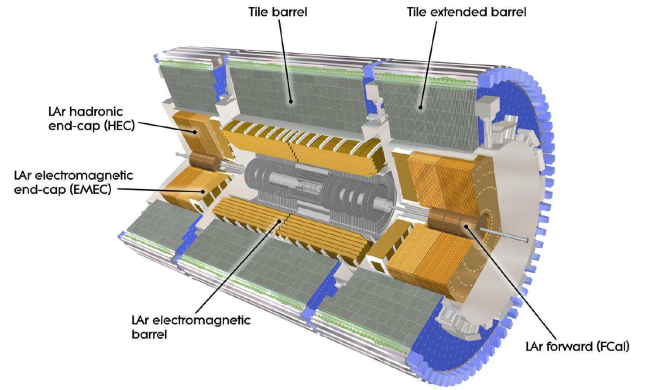


Fig. 1. ATLAS Calorimetry System [6].

As the LHC progresses toward higher luminosity operation, the number of simultaneous interactions per collision increases, making it more difficult for the trigger system to distinguish relevant physics signals from background activity. To address these limitations, the ATLAS Phase-I upgrade introduced a new digital trigger readout for the LAr calorimeter that provides finer lateral and longitudinal segmentation through Super Cells (SCs) and a wider dynamic range, improving the ability of the trigger system to resolve energy deposits in high-granularity regions and maintain performance under high pile-up conditions [7].

This work presents the development of a new firmware based on High-Level Synthesis (HLS) for the digital trigger back-end of LAr. The objective is to establish an efficient development methodology that enhances maintainability, reduces architectural complexity, and is compatible with the latency and bandwidth requirements of the Phase-I trigger system.

## II. LIQUID ARGON CALORIMETER

The ATLAS LAr calorimeter is composed of the Electromagnetic Barrel (EMB), the Electromagnetic Endcaps (EMEC), the Hadronic Endcaps (HEC), and the Forward Calorimeter (FCal). In these subsystems, the particles generate ionization currents that are collected on accordion-shaped electrodes organized into longitudinal layers. The signals are shaped and amplified in the front-end electronics to produce pulses for triggering and offline reconstruction [4].

For the Phase-I trigger system, the analog trigger towers used in the past have been replaced by Super Cells, which combine groups of calorimeter cells to provide higher spatial granularity and layer-dependent energy information to the trigger [7]. The signals from the cells are shaped and digitized at the LHC bunch crossing rate through dedicated front-end boards (FEBs) and transmitted to the digital back-end via optical links.

The digital back-end performs operations that include linearization, energy filtering, error handling, and data formatting. It also transmits SC energy measurements to the L1Calo Feature Extractors (FEXes) and provides full-granularity monitoring to TDAQ, ensuring that the operation satisfies latency, throughput, and synchronization requirements of the LHC clock. A central component of this processing chain is the LATOME (LAr Trigger Processing Mezzanine), which implements real-time firmware to validate and process the SC data that is used in the trigger decision.

## III. LATOME BOARD

The LATOME module is hosted on the LAr Carrier (LArC) blade in the backend-system. It processes the SC data from the LAr Trigger Digitizer Boards (LTDBs) at the 40 MHz bunch crossing rate and provides transverse energy ( $E_T$ ) values and quality indicators to the L1Calo FEX subsystems. Each LATOME receives 48 optical links from the LTDBs, corresponding to 320 SC channels.

The LATOME firmware, illustrated in Fig. 2, is implemented on an Intel Arria 10 FPGA and is organized as a fixed-latency pipeline operating across three principal clock domains:

- **320 MHz:** Input deserialization and initial buffering.
- **240 MHz:** Geometrical remapping and  $E_T$  computation.
- **280 MHz:** Output formatting and transmission to L1Calo.

The main processing stages are performed by four blocks:

### A. Input Stage (IS)

This block deserializes and aligns the 12-bit ADC samples from the 48 optical links, reconstructs the Bunch Crossing Identifier (BCID), validates data integrity via CRC checks, and ensures synchronization between channels.

### B. Configurable Remapping (REMAP)

Reorganizes the ADC samples of the SCs into trigger towers according to the geometry of the detector and transfers the data from 320 MHz to the processing stage at 240 MHz, applying detector-dependent mapping tables.

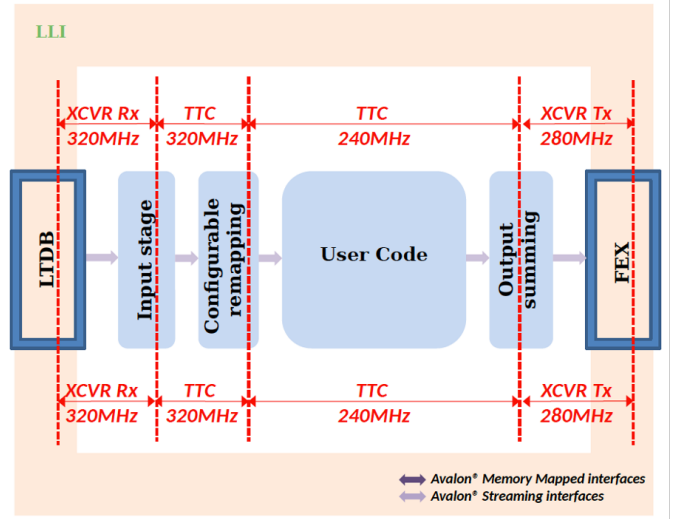


Fig. 2. Block diagram of LATOME firmware with clock domains [7].

Since the spatial distribution of SCs differs across the detector, each of the 116 LATOMEs hosts a unique remapping configuration, corresponding to its physical coverage.

### C. User Code (USER)

Implements the fixed-latency digital filtering algorithm that converts the digitized samples into calibrated  $E_T$  values and associated quality bits. It also performs timing selection to identify the correct BCID, ensuring that the reconstructed outputs meet latency requirements. The legacy version contains 62 Processing Units (PUs) to handle all SCs at 240 MHz.

### D. Output Summing (OSUM)

Performs regional  $E_T$  summations required by FEXes, applies output formatting, implements CRC checks to ensure data integrity, and transmits the data at 280 MHz. Like REMAP, the configuration for this block is also defined during initialization for each of the 116 LATOMEs, based on the detector geometry and FEX region boundaries.

To ensure proper event alignment, the entire LATOME processing pipeline operates with fixed latency and is fully synchronized to the 40 MHz LHC clock. Dedicated clock domain crossing (CDC) mechanisms are employed between IS-REMAP, REMAP-USER and USER-OSUM to maintain deterministic timing and ensure data integrity throughout the entire chain.

## IV. LATOME FIRMWARE UPGRADE

The previous operational LATOME firmware was implemented in VHDL using serial interfaces, where data remained serialized throughout the processing chain and crossed several clock domain boundaries, as illustrated in Fig 2. This time-multiplexed architecture contributed to the complexity of the firmware, for example, the REMAP stage required three sequential operations across different clock domains to align, reorder, and deliver data to the USER block.

The use of serialized data paths also increased the effort required to maintain deterministic timing and constrained the scalability of the overall design. A consequence of this architecture was the high resource utilization in USER, as 62 Processing Units were required to process the data streams. Based on the detector geometry and the optical fibers organization, the number of PUs in USER block could be reduced from 62 to 54 by implementing REMAP and OSUM with parallel interfaces rather than serialized ones. Since USER is the largest contributor to the FPGA logic consumption, this new design improves the overall resource usage.

Placing REMAP and OSUM within a common clock domain and adopting parallel data interfaces further simplifies the internal architecture. This approach improves design modularity and facilitates the future option of implementing all three main processing blocks (REMAP, USER and OSUM) to operate in a fully parallel configuration, reducing the number of clock domain crossings, lowering control overhead and easing timing closure.

To address these challenges, a new LATOME firmware architecture based on High-Level Synthesis is proposed. The HLS tool provides a higher level of abstraction for describing complex data-flow operations, enabling faster development cycles and facilitating design iteration. In the first implementation, the REMAP and OSUM blocks have been redesigned.

This work focuses on the implementation of REMAP HLS block in which the upgraded firmware replaces the serialized processing with a parallel logic structure that employs switch matrices to manage configurable routing. This redesign improves architectural clarity, enhances modularity, and establishes a more maintainable framework that supports future firmware upgrades.

In this upgraded architecture, the REMAP block is the first component to benefit from the shift to parallel processing and a unified clock domain. The redesign simplifies the data path and prepares the infrastructure for the parallelization of USER block. The following sections describe the new REMAP implementation and its switch matrix structure for the configurable data routing.

## V. REMAP HLS FIRMWARE

In the legacy firmware, the remapping process relied on sequential transfers across multiple clock domains to reorder the SC streams. This approach introduced unnecessary synchronization complexity and imposed architectural constraints on the following processing blocks. In the upgraded design, the REMAP functionality is implemented using a reconfigurable Input Switch Matrix (ISM) operating entirely within a single clock domain, avoiding time-multiplexed operations and reducing synchronization complexity.

A switch matrix is a programmable structure that connects multiple inputs to multiple outputs in parallel, according to a user-defined mapping that reflects the geometry of the calorimeter region. Fig. 3 illustrates the principle: each black dot represents a possible programmable connection between an input and an output stream.

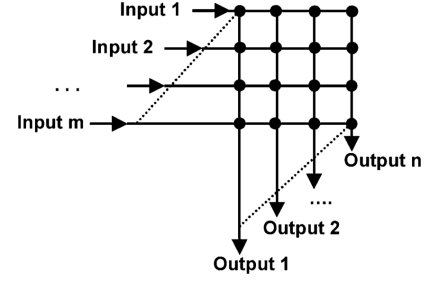


Fig. 3. Diagram of a generic switch matrix [8].

In the legacy architecture, the REMAP block receives a total of 384 input words per bunch crossing, derived from the 48 input data streams coming from the Input Stage, each carrying 8 ADC samples at 320 MHz. These serialized streams represent all Super Cells delivered by the 48 LTDB fibers. After time-multiplexed reordering and several clock domain transitions, the legacy REMAP produces 62 output data streams aligned to the 240 MHz USER clock domain, each stream carrying 6 ADC samples per bunch crossing, reflecting the expanded routing required to deliver the reordered SC groups expected by the USER block.

In the upgraded REMAP architecture, the ISM operates entirely in parallel and receives the same 384 input words per clock cycle. The ISM reduces this structure to 320 parallel outputs, corresponding to the number of SC samples required at the USER interface. The ISM therefore provides a fully configurable parallel routing layer that selects, at each cycle, the samples delivered to the USER block. To enable the use of parallel interfaces at the input and output of ISM, a deserializer and a serializer have been introduced in the REMAP block, and additional details are provided in the following sections.

### A. Initial ISM Implementation

Only the subset of connections required by a given LATOME geometry is instantiated in ISM, resulting in a sparse implementation that conserves FPGA resources while preserving full routing flexibility. The first HLS implementation of ISM, operating at 240 MHz, employed a loop-based multiplexer structure:

```

1 for i from 0 to ism_mux_size - 1 do
2   if i = ism_mux_selection then
3     data_out = data_in[i]
4   end if
5   if ism_mux_enable = 0 then
6     data_out = 0
7   end if
8 end for

```

Pseudo-code 1. Initial ISM multiplexer structure (ISM HLS v0).

Although functional, this version left the logic paths associated with unused multiplexer inputs undefined. As a result, the synthesis tool instantiated additional selection logic, leading to increased resource utilization.

### B. Optimized ISM Implementation

To reduce logic cost and improve timing closure, all multiplexer input selections were made explicit, ensuring deterministic synthesis for unused paths. The optimized HLS implementation adopts a fully defined selection structure:

```

1 if ism_mux_enable = 0 then
2   data_out = 0
3 else if ism_mux_selection <= ism_mux_size then
4   data_out = data_in[ism_mux_selection]
5 else
6   data_out = data_in[ism_mux_size]
7 end if

```

Pseudo-code 2. Optimized ISM multiplexer structure (ISM HLS v1).

The optimized design, also configured at 240 MHz, simplifies the control flow, avoids unintended logic inference, and provides a more efficient mapping to FPGA resources.

Table I summarizes the results obtained for ISM in the initial HLS implementation (HLS v0) and the optimized HLS version (HLS v1), both designed as combinational circuits. The optimized version achieves a substantial reduction in Adaptive Logic Modules (ALMs) and improved maximum frequency while maintaining identical throughput and latency.

TABLE I  
ISM RESOURCE UTILIZATION

| Version | Latency | Throughput | ALMs  | Slack   | Fmax    |
|---------|---------|------------|-------|---------|---------|
| HLS v0  | 4,16 ns | 1 cycle    | 26928 | 1,80 ns | 151 MHz |
| HLS v1  | 4,16 ns | 1 cycle    | 21856 | 2,30 ns | 293 MHz |

### C. Design Exploration of the ISM Block

High-Level Synthesis enables rapid architectural exploration by modifying Catapult directives rather than performing manual RTL redesign. For the ISM HLS block with the optimized multiplexer structure, different implementation styles were evaluated by adjusting directives that control pipelining, loop transformations, resource sharing, and module inlining, enabling systematic performance–area trade-offs without altering the functional behavior of the code. These directives also determine how the ISM logic is structurally realized in hardware, influencing the balance between parallelism, critical-path length, and resource usage. The multiplexer logic used in these variants is shown in Pseudo-code 2, and a simplified ISM HLS top-level is presented in Pseudo-code 3, to illustrate how the multiplexers are instantiated in parallel for each of the 320 output channels at 240 MHz.

```

1 # pragma: design style combinational
2 # pragma: unroll factor 320
3 for n from 0 to output_streams - 1 do
4   sel = selection_signal[n]
5   ena = enable_signal[n]
6   x_in = input_streams_array
7   y[n] = mux(x_in, sel, ena) # mux in pseudo-code 2
8 end for

```

Pseudo-code 3. Multiplexer instantiation in the ISM HLS top-level.

To characterize the architectural space, three main circuit organizations for ISM were generated by configuring it on Line 1 of Pseudo-Code 3: (i) *inlined* implementations, where hierarchy boundaries are removed and the multiplexer logic is merged into the top-level module to reduce interface overhead; (ii) *sequential* implementations, where pipeline stages are introduced to reduce combinational depth; and (iii) *combinational* implementation, where the ISM logic is synthesized as a single cycle of purely combinational routing. These variants were evaluated to ensure the ISM meets timing at 240 MHz. They differ in logic depth, routing complexity, and the amount of local control logic inferred by the tool. The resulting performance and resource metrics obtained for each configuration are summarized in Table II.

TABLE II  
ISM HLS v1 WITH CONFIGURABLE PRAGMA STYLES

| Pragma        | Latency | Throughput | ALMs  | Slack   | Fmax    |
|---------------|---------|------------|-------|---------|---------|
| Inline        | 4,16 ns | 1 cycle    | 21856 | 2,30 ns | 293 MHz |
| Sequential    | 8,32 ns | 2 cycles   | 21821 | 2,30 ns | 291 MHz |
| Combinational | 4,16 ns | 1 cycle    | 21856 | 2,30 ns | 293 MHz |

Loop-unrolling studies were also performed in the HLS top level to explore different parallelism and resource-sharing configurations. With full unrolling (unroll factor of 320), the design instantiates one multiplexer for each of the 320 output streams, achieving a latency of a single clock cycle at the cost of the highest area. Intermediate unroll factors share hardware across multiple streams; for example, an unroll factor of 160 uses one block for every two streams, resulting in a latency of two clock cycles, while smaller factors increase sharing and latency accordingly. The full unroll was adopted in the proposed design of REMAP to have a latency of 1 clock cycle.

### D. Implementation of SERDES for the REMAP Interface

The ISM HLS operates with fully parallel input and output interfaces, but the interfaces around REMAP in the LATOME firmware remain serialized, since the IS delivers SC samples in a serialized stream, and the USER block expects data in a similar format. To interface the parallel ISM architecture with these serialized domains, dedicated serializer-deserializer (SERDES) blocks are required.

A SERDES block converts data between serial and parallel representations by grouping incoming serial samples into parallel words, or by streaming parallel data out sequentially at the required rate. This section describes the deserializer implemented at the REMAP input and the serializer developed for its output, enabling the integration of the REMAP HLS block into the LATOME firmware.

1) *Serial to Parallel Converter*: The deserialization stage is implemented through a serial-to-parallel (S2P) converter that reconstructs each group of *NWORDS* samples of width *NBITS* into a parallel word suitable for the ISM interface. Each serial stream is accompanied by a Start-of-Packet (SOP) flag, which is asserted together with the first serial word of a packet and is used to synchronize the capture of a complete parallel word.

For clarity, Pseudo-code 4 illustrates the behavior of the S2P logic for a single input stream. In the full REMAP implementation, this converter is instantiated 48 times in parallel, which is required for the  $48 (\text{links}) \times 8 (\text{words per stream}) = 384$  incoming data streams, providing the parallel data format required by the ISM block for each BCID.

```

1 # NWORDS entries with width NBITS
2 initialize shift_reg[0 .. NWORDS-1]
3 initialize shadow_reg[0 .. NWORDS-1]
4 on each rising edge of clk:
5     # shift existing samples
6     for i from 0 to NWORDS-2 do
7         shift_reg[i] = shift_reg[i+1]
8     end for
9     # load new serial sample of width NBITS
10    shift_reg[NWORDS-1] = data_in
11    # capture a full parallel word when SOP arrives
12    if sop_in = 1 then
13        shadow_reg = shift_reg
14    end if
15    # update SOP output
16    sop_out = sop_in
17 # parallel output word of NWORDS*NBITS bits
18 data_out = shadow_reg

```

Pseudo-code 4. Logic of the serial-to-parallel (S2P) converter.

2) *Parallel to Serial Converter*: The serialization stage is implemented through a parallel-to-serial (P2S) converter that transforms each parallel word from the *NWORDS* samples of width *NBITS* into a sequential stream suitable for transmission toward the USER block. The converter sequentially emits the samples of the parallel word for each BCID across consecutive clock cycles, restoring the serial format expected by USER.

As in the deserialization stage, a SOP flag is used to indicate the arrival of each parallel word, that is stable and unchanged during each BCID (25 ns). In the P2S converter, the SOP is asserted together with the first serial output sample, allowing the data for USER block to remain synchronized with the SOP.

The Pseudo-code 5 illustrates the behavior of the serializer for a single output stream. In the full REMAP implementation, this block is instantiated in parallel 54 (streams)  $\times$  6 (words per stream) = 324 for all 320 outputs produced by the ISM block. The extra connections in the last serializer (324–320 = 4) are grounded because only 320 outputs are in use.

```

1 # NWORDS entries with width NBITS
2 initialize shifter[0 .. NWORDS-1]
3 initialize index = 0
4 # load new parallel word using the SOP flag
5 if sop_in = 1 then
6     shifter = data_in # parallel input
7     index = 0
8 end if
9 # on each rising edge of clk:
10 data_out = shifter[index] # emit one NBITS word
11 if index = 0 then
12     sop_out = 1 # SOP on first serial word
13 else
14     sop_out = 0 # SOP for other serial words
15 end if
16 # advance index for next cycle
17 if index < NWORDS - 1 then
18     index = index + 1
19 end if

```

Pseudo-code 5. Logic of the parallel-to-serial (P2S) converter.

It should be noted that this serializer will no longer be required once the USER block is upgraded to operate with parallel interfaces. Its included in the current implementation to preserve compatibility with the existing serial interfaces surrounding the REMAP block.

#### E. Clock Domain Crossing

The REMAP block receives data from the IS at 320 MHz, while the USER block operates in the 240 MHz clock domain. A clock domain crossing is done after the deserialization, where the serial streams have been converted into parallel data. The transfer between these domains uses a SOP flag, which provides a deterministic alignment point for capturing the parallel word in the slower clock domain. The SOP flag from IS to REMAP (at 320 MHz) is used to produce a corresponding SOP signal in the 240 MHz domain.

For this purpose, the SOP Generator block has been implemented to resynchronize the incoming SOP to the 240 MHz clock, and the SOP Phase generator block was created to adjust the phase of the signal, ensuring that the transfer of the data occurs at the center of the 25 ns interval window. During this window, the parallel data remains unchanged. The diagram for the SOP generation scheme is illustrated in Fig. 4.

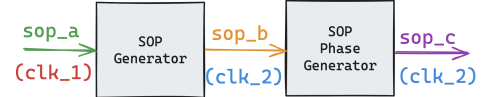


Fig. 4. Diagram of the SOP Generation Scheme.

1) *SOP Generator*: The block used to generate the SOP flag, illustrated in Fig. 5, implements a safe and deterministic transfer of the SOP signal from the 320 MHz clock domain (*clk1*) to the 240 MHz clock domain (*clk2*). The SOP must be transferred with precise timing and without metastability risks for the alignment point for the clock domain crossing inside REMAP. The circuit achieves this by toggling the SOP, synchronizing it across domains, and detecting its edge on the receiving side.

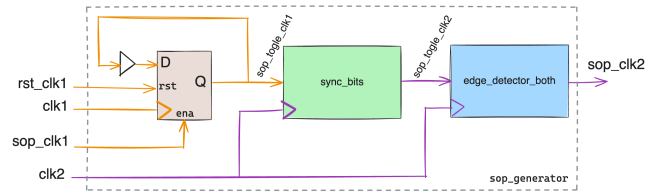


Fig. 5. SOP Generator Circuit.

2) *SOP Phase Generator*: This block provides a programmable phase shift for the SOP signal in the 240 MHz domain, ensuring that the clock domain crossing occurs at a well-controlled point within the 25 ns interval in which the parallel data remain stable. The circuit operates by passing the incoming SOP pulse through a tapped delay line and selecting one of its stages according to a signal control.



The selected tap produces a synchronized reset for a circular shift register and the rotation defines the final SOP phase. In this manner, the firmware can align the SOP pulse to the desired clock edge, allowing the clock transfer of the data from the 320 MHz to the 240 MHz clock domain in REMAP.

#### F. Firmware Verification and Validation

The complete REMAP architecture, including the SERDES and SOP generation blocks, is shown in Fig. 6. The serialized SC data and the associated SOP flag arrive from the IS at 320 MHz and are converted to parallel at the REMAP input. The SOP signal is transferred to the 240 MHz domain through dedicated generation and phase-adjustment logic, which aligns it to the center of the 25 ns interval window with the stable parallel data. The resulting 384 parallel streams are then processed by the ISM block, producing 320 reordered outputs that are subsequently serialized and sent to USER.

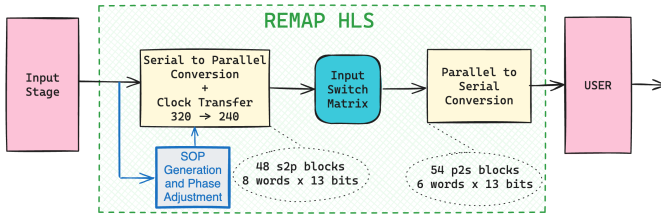


Fig. 6. REMAP Diagram.

1) *HLS C++ Simulation*: The standalone ISM HLS block was first validated using a C++ testbench for the functional verification of the remapping logic and allowed quick iterations over the studies with different configurations, such as pipelining, inlining and resource sharing, before moving to RTL-level verification.

2) *Python Cocotb RTL Verification*: The RTL verification was performed using Cocotb to validate the full REMAP interface, including the SERDES, ISM HLS, SOP generation logic, and the clock domain crossing. These testbenches confirmed the correct SOP alignment and verified that the ISM block received stable and properly synchronized parallel data.

3) *Laboratory Tests for SOP Calibration*: The hardware validation was performed in the Electronics Monitoring Facility (EMF) at CERN to confirm the SOP timing calibration under real operating conditions for the firmware in a LATOME board, where the phase settings derived from simulation have been tested with real clocks and links.

#### VI. RESULTS

The use of HLS for the parallel REMAP facilitated design exploration and verification, allowing architectural changes to be introduced through synthesis directives without modifying the source code or altering the module interfaces. As a result, the testbenches could be reused for all circuit configurations for consistent functional evaluation throughout the design-space analysis. It was possible to generate architectures by changing directives from fully parallel to highly shared implementations, keeping the functional behavior.

This approach provided clear traceability between directive settings and hardware cost, revealing how each configuration affected critical-path length, frequency, and logic cost.

HLS also accelerated functional verification at the algorithmic level. For example, the C++ testbench for REMAP processing 10 000 bunch crossings completed  $\approx 29\times$  faster than the equivalent RTL cocotb testbench, as shown in Table III. This runtime difference enabled rapid iterations during early development while preserving the ability to validate the proper functionality of the blocks.

TABLE III  
REMAP SIMULATION TIME FOR 10 000 BCIDS

| Simulation Type      | Execution Time |
|----------------------|----------------|
| HLS C++ Testbench    | 2,13 s         |
| RTL Cocotb Testbench | 62,34 s        |

Laboratory tests confirmed reliable SOP capture and correct routing of all 384 channels through ISM. Accurate SOP calibration is essential, as any misalignment would corrupt the data during the transfer between clocks. The tests also verified that all channels were correctly routed through REMAP with behavior consistent with expected by the RTL simulations.

#### VII. CONCLUSIONS

The upgraded REMAP HLS has fully replaced the legacy VHDL implementation and is now operational in the ATLAS LAr trigger readout system at CERN. The parallel architecture implemented in REMAP reduced logic complexity and allowed a decrease in the number of PUs in the USER block from 62 to 54, reducing overall firmware logic from 290k to 270k ALMs (about a 7% reduction) and reducing the total FPGA utilization from 69% to 64%. Ongoing work on the OSUM and USER blocks is continuing this modernization effort, ensuring that the LATOME firmware architecture evolves toward a robust and maintainable long-term solution.

#### ACKNOWLEDGMENT

This work was supported by CNPq. We also extend our gratitude to the ATLAS Experiment and LAr team.

#### REFERENCES

- [1] Evans, L., and Bryant, P., "LHC Machine", Journal of Instrumentation, IOP Publishing, vol. 3, no. 08, p. S08001, 2008.
- [2] Aad, G. *The ATLAS Experiment at the CERN Large Hadron Collider*. JINST, vol. 3, pp. S08003, 2008.
- [3] Carrio, F., Kim, H. Y., Moreno, P., Reed, R., Sandrok, C. "Design of an FPGA-Based Embedded System for the ATLAS Tile Calorimeter Front-End Electronics Test-Bench." CERN Document Server, ATL-TILECAL-PROC-2013-017, Geneva, 2013.
- [4] Aleksa, M., Cleland, W., Enari, Y. "ATLAS Liquid Argon Calorimeter Phase-I Upgrade Technical Design Report." CERN Document Server, CERN-LHCC-2013-017, Geneva, 2013.
- [5] THE ATLAS HLT/DAQ/DCS GROUP. "ATLAS High-Level Trigger, Data Acquisition and Controls." ATLAS Technical Report, CERN, 2001.
- [6] Pequeno, J. "Computer Generated Image of the ATLAS Calorimeter." CERN Document Server, CERN-GE-0803015, 2008.
- [7] ATLAS Collaboration, "The Phase-I Trigger Readout Electronics Upgrade of the ATLAS Liquid Argon Calorimeters," JINST, P05003, 2022.
- [8] Kirichenko, A., Gupta, D., and Sarwana, S. "RSFQ asynchronous switch matrix," IEEE Transactions on Applied Superconductivity, vol. 17, pp. 505–507, 2007.